



Sprawozdanie techniczne - BAI Security Lab

1. Informacje o projekcie

Projekt realizuje aplikacje webowa w modelu Vulnerable vs Secure. Aplikacja jest jedna, ale jej zachowanie zależy od przełącznika `SECURITY_ENABLED`.

- `SECURITY_ENABLED=false` - tryb podatny, celowo dopuszcza ataki do demonstracji.
- `SECURITY_ENABLED=true` - tryb bezpieczny, ten sam scenariusz powinien zostać zablokowany.

Zespół jest 2-osobowy, więc wymagany zakres to:

- 2 podatności obowiązkowe: SQL Injection, XSS.
- 3 podatności dodatkowe zgodnie z formułą $2n - 1$.

Projekt pokazuje szerszy zakres: 2 podatności obowiązkowe + 6 dodatkowych.

2. Architektura i technologie

Stos technologiczny:

- Go + Gin - backend HTTP i routing.
- SQLite - lokalna baza danych.
- Templ - server-side HTML templates.
- HTMX - częściowe odświeżanie widoków.
- Tailwind CSS - warstwa stylów.

Główne pliki:

- `main.go` - konfiguracja routera, rejestracja endpointów i włączenie trybu security.
- `internal/config/config.go` - odczyt `DB_PATH`, `PORT`, `SECURITY_ENABLED`.
- `internal/db/db.go` - migracje i dane seed.
- `internal/service/service.go` - logika biznesowa i dostęp do bazy.

- `internal/handlers/handlers.go` - handlers JSON/UI i roznice vulnerable/secure.
- `internal/views/pages.tpl` - widoki UI i strony demo.
- `main_integration_test.go` - testy integracyjne scenariuszy atak/obrona.

3. Finalny katalog podatnosci

#	Podatnosc	Wymagalnosc	Status	Demo
1	SQL Injection	obowiazkowa	gotowe	<code>/ui/search</code> , <code>/api/search</code> , <code>/api/search-vulnerable</code>
2	Stored XSS	obowiazkowa	gotowe	<code>/ui/posts/view/1</code> , <code>/api/comments-vulnerable</code>
3	Broken Authentication	dodatkowa	gotowe	<code>/ui/login</code> , <code>/login</code>
4	Broken Access Control / IDOR	dodatkowa	gotowe	<code>/ui/idor-demo</code> , <code>DELETE /posts/:id</code>
5	CSRF	dodatkowa	gotowe	<code>/ui/csrf-demo</code> , <code>/ui/csrf-secure</code>
6	Sensitive Data Exposure	bonus	gotowe	<code>/ui/db-expose</code> , SQLite <code>users.password_hash</code>
7	Path Traversal / LFI	bonus	gotowe	<code>/ui/path-traversal</code> , <code>/api/files-*</code>
8	Command Injection	bonus	gotowe	<code>/ui/cmd-injection</code> , <code>/api/ping-*</code>

3.1. Zrzuty ekranow z aplikacji

Zrzuty ekranow znajduja sie w katalogu `docs/screenshots/` i pokazuja porownanie trybu vulnerable oraz secure po uzyciu przelacznika w naglowku.

Plik	Co pokazuje
docs/screenshots/01-vuln-demos-vulnerable.jpg	Hub podatnosci w trybie vulnerable
docs/screenshots/02-sqli-vulnerable-results.jpg	SQL Injection: payload zwraca wyniki w trybie vulnerable
docs/screenshots/03-path-traversal-vulnerable.jpg	Path Traversal: odczyt pliku spoza uploads
docs/screenshots/04-command-injection-vulnerable.jpg	Command Injection: wykonanie dodatkowej komendy
docs/screenshots/05-vuln-demos-secure-after-toggle.jpg	Hub po przełączeniu przyciskiem na tryb secure
docs/screenshots/06-sqli-secure-blocked.jpg	SQL Injection: payload nie przechodzi przez prepared statements
docs/screenshots/07-path-traversal-secure-blocked.jpg	Path Traversal: ../ zostaje odrzucone
docs/screenshots/08-command-injection-secure-blocked.jpg	Command Injection: host z separatorem komend zostaje odrzucony

Przelacznik trybu jest widoczny w naglowku aplikacji jako `↔ Secure` albo `↔ Vulnerable` . Technicznie wysyla `POST /ui/mode/toggle` , aktualizuje tryb handlerow i service layer, a

potem wraca na aktualna strone demo.

4. SQL Injection

Opis:

SQL Injection polega na potraktowaniu danych uzytkownika jako fragmentu zapytania SQL. Atakujacy moze zmienic warunek `WHERE`, ominac filtrowanie albo wykonac `UNION SELECT`.

PoC vulnerable:

```
curl "http://localhost:8080/api/search?q=' OR 1=1 --"
curl "http://localhost:8080/api/search-vulnerable?q=zz' UNION SELECT id, username, password"
```

Efekt:

W trybie vulnerable endpoint zwraca wiecej rekordow niz powinien, w tym dane z tabeli `users` przy payloadzie `UNION`.

Remediacja:

W trybie secure wyszukiwanie uzywa parametrow `?`, a input jest traktowany jako literalny wzorzec `LIKE`, nie jako SQL.

Kod:

- Vulnerable: `internal/service/service.go` - `SearchPostsVulnerable`.
- Secure: `internal/service/service.go` - `SearchPostsSecure`.
- Testy: `main_integration_test.go` - `TestIntegration_SearchSQLi_VulnerableMode`, `TestIntegration_SearchSQLi_SecureMode`.

5. Stored XSS

Opis:

Stored XSS polega na zapisaniu zlosliwego HTML/JavaScript w bazie i pozniejszym wykonaniu go u innych uzytkownikow przy renderowaniu strony.

PoC vulnerable:

```
curl -X POST http://localhost:8080/api/comments-vulnerable \
-H "Content-Type: application/json" \
-d '{"post_id":1,"body":"<script>alert(\"XSS\")</script>","author\":\"attacker\"}'
```

Mozna tez uzyc UI:

```
/ui/posts/view/1
payload: <img src=x onerror="alert(1)">
```

Efekt:

W trybie vulnerable komentarz jest zapisany i renderowany jako raw HTML.

Remediacja:

W trybie secure komentarz jest czyszczony z `script`, event handlerow `on*=` i `javascript:`, a nastepnie escapowany. Widok secure renderuje tresc jako tekst.

Kod:

- Vulnerable render: `internal/views/pages.tpl` - `CommentsList` z `templ.Raw`.
- Secure storage: `internal/service/service.go` - `CreateComment`, `stripUnsafeHTML`.
- Secure seed: `internal/db/db.go` - `encodeCommentForSeed`.
- Testy: `main_integration_test.go` - `TestIntegration_StoredXSS_VulnerableMode`, `TestIntegration_StoredXSS_SecureMode`.

6. Broken Authentication

Opis:

Broken Authentication oznacza, ze aplikacja blednie weryfikuje tozsamosc uzytkownika. W projekcie vulnerable mode akceptuje dowolne haslo dla istniejacego loginu.

PoC vulnerable:

```
curl -i -X POST http://localhost:8080/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"dowolne-haslo"}'
```

Efekt:

W trybie vulnerable serwer zwraca sukces i ustawia cookie sesji.

Remediacja:

W trybie secure aplikacja porownuje haslo przez bcrypt i blokuje bledne dane. Dodany jest tez prosty rate limit po nieudanych probach.

Kod:

- Handler: `internal/handlers/handlers.go` - `evaluateLogin` .
- Service: `internal/service/service.go` - `ValidateUserCredentials` , `CheckRateLimit` , `RecordLoginFailure` .
- Seed hasel: `internal/db/db.go` - `encodePasswordForSeed` .

7. Broken Access Control / IDOR

Opis:

Broken Access Control wystepuje, gdy zalogowany uzytkownik moze wykonac operacje na zasobie, ktorego nie jest wlascicielem. W projekcie pokazujemy usuwanie postu po samym ID.

PoC vulnerable:

1. Uruchom vulnerable mode.
2. Zaloguj sie jako `user1 / anything` .
3. Wejdz na `/ui/idor-demo` .
4. Usun post nalezacy do `admin` .

Efekt:

W trybie vulnerable wystarczy samo zalogowanie, wlasciciel posta nie jest sprawdzany.

Remediacja:

W trybie secure handler sprawdza, czy aktualny uzytkownik jest autorem posta albo adminem.

Kod:

- Handler: `internal/handlers/handlers.go` - `PostDelete` , `PagePostDelete` ,

`canDeletePost` .

- Service: `internal/service/service.go` - `GetPostAuthor` , `IsUserAdmin` .
- Testy: `main_integration_test.go` -
`TestIntegration_DeleteAuthorization_SecurityEnabled` .

8. CSRF

Opis:

CSRF polega na wymuszeniu przez obca strone wykonania akcji w aplikacji, w ktorej ofiara jest zalogowana. Przegladarka automatycznie dolacza cookie sesji, wiec sam login nie chroni przed takim atakiem.

PoC vulnerable:

```
<form method="POST" action="http://localhost:8080/ui/csrf-demo">
  <input name="new_email" value="hacked@evil.com">
</form>
<script>document.forms[0].submit()</script>
```

Efekt:

W trybie vulnerable email zalogowanego uzytkownika moze zostac zmieniony bez tokena.

Remediacja:

Secure route `/ui/csrf-secure` generuje token, zapisuje go w cookie `bai_csrf_token` , umieszcza w ukrytym polu formularza i sprawdza zgodnosc przy POST.

Kod:

- Vulnerable: `internal/handlers/handlers.go` - `CsrfFormVulnerable` .
- Secure: `internal/handlers/handlers.go` - `CsrfSecureForm` , `generateCSRFToken` .
- Widok: `internal/views/pages.tpl` - `CSRFDemoPage` .

9. Sensitive Data Exposure

Opis:

Sensitive Data Exposure polega na ujawnieniu danych wrażliwych albo przechowywaniu ich w

niebezpiecznej formie. W projekcie vulnerable mode zapisuje hasła jawnie w kolumnie `password_hash`.

PoC vulnerable:

```
sqlite3 app.db "SELECT username, password_hash, email FROM users;"
```

Albo przez UI:

```
/ui/db-expose
```

Efekt:

W vulnerable mode kolumna `password_hash` zawiera faktyczne hasło, np. `admin`.

Remediacja:

W secure mode hasła są zapisywane przez bcrypt. Nawet wyciek bazy nie ujawnia haseł wprost.

Kod:

- Service: `internal/service/service.go` - `preparePassword`.
- Seed: `internal/db/db.go` - `encodePasswordForSeed`.
- UI: `internal/views/pages.tpl` - `DBExposePage`.

10. Path Traversal / LFI

Opis:

Path Traversal polega na manipulacji ścieżką pliku, np. przez `../`, aby wyjść poza katalog dozwolony i odczytać pliki aplikacji lub systemu.

PoC vulnerable:

```
curl "http://localhost:8080/api/files-vulnerable?name=../go.mod"
curl "http://localhost:8080/api/files-vulnerable?name=../app.db" | xxd | head -4
```

Efekt:

Vulnerable endpoint czyta plik spoza katalogu `uploads` .

Remediacja:

Secure endpoint odrzuca sciezki absolutne i `..` , normalizuje sciezke oraz sprawdza, czy wynik pozostaje pod `./uploads` .

Kod:

- Vulnerable: `internal/handlers/handlers.go` - `FilesVulnerable` .
- Secure: `internal/handlers/handlers.go` - `FilesSecure` , `safeUploadPath` .
- UI: `internal/views/pages.html` - `PathTraversalPage` .
- Testy: `main_integration_test.go` - `TestIntegration_PathTraversal_LFI` .

11. Command Injection

Opis:

Command Injection wystepuje, gdy input uzytkownika trafia do shella i moze zawierac metaznaki, takie jak `;` , `&&` albo `|` .

PoC vulnerable:

```
curl "http://localhost:8080/api/ping-vulnerable?host=127.0.0.1%3B%20whoami"
curl "http://localhost:8080/api/ping-vulnerable?host=127.0.0.1%3B%20id"
```

Efekt:

Vulnerable endpoint wykona dodatkowa komende po separatorze `;` .

Remediacja:

Secure endpoint waliduje host allowlista `^[a-zA-Z0-9.\-]+$` i uruchamia `exec.Command("ping", "-c1", host)` bez shella.

Kod:

- Vulnerable: `internal/handlers/handlers.go` - `PingVulnerable` .
- Secure: `internal/handlers/handlers.go` - `PingSecure` , `validHostRE` .
- UI: `internal/views/pages.html` - `CmdInjectionPage` .
- Testy: `main_integration_test.go` - `TestIntegration_CommandInjection` .

12. Checklista bezpieczeństwa

Zastosowane mechanizmy w trybie secure:

- Prepared statements / placeholders ? dla SQL.
- Bcrypt dla hasel.
- HttpOnly cookie dla sesji.
- HMAC podpis cookie w secure mode.
- Kontrola autoryzacji owner/admin przy usuwaniu postow.
- CSRF token dla chronionego formularza.
- Escaping i sanityzacja komentarzy.
- Walidacja sciezek plikow.
- Walidacja argumentow komendy i brak shella.

13. Komendy walidacyjne

```
go test -tags=integration .  
go test ./...  
go build -o /tmp/bai_check_app main.go
```

Aktualny wynik: wszystkie powyzsze komendy przechodza.